



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И
УПРАВЛЕНИЯ

ДОМАШНЯЯ РАБОТА № 3

«Разработка приложения на языке С# с базой
данных и пользовательским интерфейсом»

ДИСЦИПЛИНА: «Архитектура программных систем»

Выполнил: студент гр. ИУ5-21Б

_____ (Постнов Ю.С.)
(Подпись) (Ф.И.О.)

Проверил:

_____ (Гапанюк Ю. Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

2026 г.

Цель работы: Разработка веб-приложения для ведения справочника издательств и журналов с использованием современных технологий .NET и Entity Framework Core.

Задачи:

1. Спроектировать базу данных (SQLite) и реализовать доступ к ней через EF Core (Code First).
2. Реализовать CRUD-операции для сущностей “Издательство” и “Журнал”.
3. Разработать систему отчётов с использованием LINQ для агрегации данных.
4. Обеспечить корректную работу веб-сервера (Kestrel) с поддержкой конфигурации через аргументы командной строки.
5. Выполнить рефакторинг кода для соответствия современным стандартам .NET 10.

1. Изменения кода

Рефакторинг точки входа (Program.cs)

Первоначальная версия использовала низкоуровневый HostBuilder, что затрудняло интеграцию со стандартными механизмами конфигурации и логирования. Код был переписан на современный WebApplication API.

До изменения (фрагмент):

```
var host = new HostBuilder()  
    .ConfigureWebHost(webBuilder => {  
        webBuilder.UseKestrel()  
            .UseUrls(urls)  
            // Ручная настройка сервисов и конвейера...  
    })  
    .Build();
```

После изменения:

```
var builder = WebApplication.CreateBuilder(args);  
  
// Автоматическая регистрация сервисов  
builder.Services.AddControllersWithViews();  
builder.Services.AddDbContextFactory<AppDbContext>(options =>  
  
options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));  
  
var app = builder.Build();  
// Настройка конвейера через стандартные методы Use...  
app.Run();
```

Пояснение: Использование `WebApplication.CreateBuilder(args)` позволяет приложению автоматически считывать параметры командной строки (например, `--urls`), подключать файлы конфигурации (`appsettings.json`) и настраивать детализированное логирование в консоль “из коробки”.

Реализация логики отчётов (`ReportsController.cs`)

Для формирования отчётов использовались возможности LINQ to Entities, что позволяет выполнять группировку и агрегацию данных на стороне СУБД.

```
public IActionResult Index()
{
    using var context = _dbContextFactory.CreateDbContext();

    // Средний тираж по издательствам
    var averageCirculationByPublisher = context.Magazines
        .GroupBy(magazine => magazine.Publisher!.Name)
        .Select(group => new PublisherAverageReportRow
        {
            PublisherName = group.Key,
            AverageCirculationK = group.Average(m =>
m.CirculationK)
        })
        .OrderByDescending(row => row.AverageCirculationK)
        .ToList();

    // ... передача данных во View
}
```

Пояснение: Группировка `GroupBy` и расчет среднего значения `Average` выполняются эффективно, преобразуясь в SQL-запрос.

2. Вывод программы в консоль

При запуске приложения программа выводит следующую информацию:

```
/Users/xserx/код/bmstu/AAS0IU/Homework3/ConsoleApp4/ConsoleApp4/bin/Debug/net10.0/ConsoleApp4
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (4ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT COUNT(*) FROM "sqlite_master" WHERE "type" = 'table' AND "rootpage" IS NOT NULL;
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (0ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT EXISTS (
        SELECT 1
        FROM "publishers" AS "p")
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /Users/xserx/код/bmstu/AAS0IU/Homework3/ConsoleApp4/ConsoleApp4
```

Вывод

В ходе работы было успешно разработано и отлажено ASP.NET Core MVC приложение. Переход на современный WebApplication API позволил решить проблемы с HTTP-подключением и обеспечил гибкость настройки сервера. Реализованная архитектура (разделение на контроллеры, модели и вью-модели) обеспечивает масштабируемость и легкость поддержки приложения. Использование Entity Framework Core позволило абстрагироваться от SQL-запросов, сохранив при этом высокую производительность при формировании аналитических отчётов.